

Homework 4

Due: Saturday, October 26th, 11pm

While problem sets 1–3 focused on learning to debug, the next few will focus on how to write code in such a manner that debugging occurs much less frequently (ideally, not at all) because the code we write is known to be correct *before* we run it for the first time.

With that goal in mind, the next few assignments will be split into separate written and coding parts, where the written part is to be completed *before* starting on the coding part.

This worksheet contains only the *written* parts of HW4; the coding instructions will be released Thursday (10/24) and be due Monday (10/28).

Submission

After completing all parts below, submit your solutions on Gradescope. The collection of all written answers to problems described in this worksheet should be submitted as a PDF to **“HW4 Written”**.

Don't forget to check that the submitted file is up-to-date with all written work you completed!

You may handwrite your work (on a tablet or paper) or type it, provided it is legible and dark enough to read. When you turn in on Gradescope, please match each HW problem to the page with your work on it. If you fail to have readable work or assign pages, you will receive a point deduction.

Task 1 – Off the Beaten Math

[16 pts]

For each of the following functions, translate the code into a function definition written in our math notation, using pattern matching. Unless the comments above the code say otherwise, you *can* assume that any value of the declared TypeScript type is allowed by the function.

Make sure your rules are *exclusive* and *exhaustive*!

```
(a) const a = (o: boolean, t: [bigint, bigint]): [bigint, bigint] => {
    const [i, j]: [bigint, bigint] = t;
    if (o) {
        return [i + 1n, j];
    } else {
        return [j + 1n, i];
    }
};
```

```
(b) const b = (s: bigint, o: boolean): List<bigint> => {
    if (o) {
        return cons(s, cons(-s, nil));
    } else {
        return cons(-s, cons(s, nil));
    }
};
```

```
(c) // s and t allow only non-negative integers
const c = (r: {s: bigint, t: [bigint, bigint]}): bigint => {
    const [i, j]: [bigint, bigint] = r.t;
    if (r.s === 0n) {
        return i;
    } else if (r.s === 1n) {
        return i + 1n;
    } else {
        return j + c({s: r.s - 1n, t: r.t});
    }
};
```

Task 2 – Pell on Wheels

[14 pts]

In this problem, we will practice a skill that you will use a lot in coming assignments. We'll start with an English description and formalize that description into a mathematical definition which could serve as a specification if we were to write this in TypeScript.

We'll be converting an English description of a function that calculates **Pell numbers** to math notation. We'll denote a Pell number as $p(n)$ for some non-negative integer n . We define $p(0) = 0$ and $p(1) = 1$. After that, we define $p(n)$ to be the sum of twice the previous Pell number and the Pell number before that. With that definition in hand, our goal is to write a function " $s(n)$ " that gives the sum of the *first* n Pell numbers. For example, $s(2) = p(0) + p(1) = 1$.

For reference, here are the first five Pell numbers and their sums:

n	$p(n)$	$s(n)$
0	0	0
1	1	0
2	2	1
3	5	3
4	12	8
5	29	20

- (a) The description above is in English, so our first step is to formalize it into a math notation.

Define two mathematical functions, " p " and " s ", both taking non-negative integers as input. Define each function recursively with pattern matching.

- (b) Show how your mathematical definition would execute $s(6) = 49$ by writing out the sequence of recursive calls. Include the arguments and what is returned for each recursive call.

Use any sensible notation to clearly show the sequence of calls. (If a call is made a second time, you can reuse the result of that call without showing all recursive calls that would be made.)

Task 3 – Good To the Last Swap

[14 pts]

In this problem, we will create some functions that allow us to encode and decode secret string messages.

Our messages will be represented as lists of integers, where each integer is in the range 0–25 and the integer j represents the j -th Latin letter. For example, 'a' = 0, 'b' = 1, and 'z' = 25.

Let's start by discussing how we encode individual characters.

We will say that “negating” a range of characters means swapping the j -th character from the beginning with the j -th character from the end. For example, negating the range 0–25 turns 'a' (0) into 'z' (25), 'c' (2) into 'x' (23), etc.

Note that negating a range twice gets us back to where we started, so if we negate ranges as our cipher, we can write one function that both encodes and decodes!

Our encoding function will be a little more complicated than just that. It will negate the four ranges consisting of the first 6 characters (0–5), the next 7 characters (6–12), the next 6 characters (13–18), and last 7 characters (19–25), each individually. After negating these four ranges, it will also swap the first and third ranges and the second and fourth ranges. (Repeating a swap also gets us back to where we started, so again this one function will both encode and decode.)

To encode a message, which is a list of characters, we apply the character encoding individually to each character in the list.

- (a) Above, we were given an English definition of the problem, so our first step is to formalize it.

The function “ns” will take an integer the range 0–25 as input and returns the value produced by *negating and swapping* ranges as described above. For integer values outside the range 0–25, we define ns to leave those unchanged.

- (b) Now, given “ns” defined in part (a), formalize “cipher”, which encodes messages.

Write a formal definition using recursion. Assume that the mathematical function “ns” defined in part a) correctly implements the behavior described above.

Before we can get to the next two problems, we need the following mathematical definitions.

Blocks

In this assignment, we will write some math that displays mazes. Each maze is made up of Blocks. Mathematically, each block is a record of the following type:

$$\text{type Block} := \{\text{form} : \text{STRAIGHT}, \text{color} : \text{Color}, \text{direction} : \text{Line}\} \\ | \{\text{form} : \text{ANGLED}, \text{color} : \text{Color}, \text{direction} : \text{Corner}\}$$

Individual Blocks include a color property, which are elements of the following type:

$$\text{type Color} := \text{BLUE} \mid \text{ORANGE}$$

Blocks also include a direction property that describes how the block is oriented (i.e. how it is rotated). direction is defined with *different types* depending on the form property of the block.

STRAIGHT Blocks contain a straight line that spans the block in 1 direction, either top to bottom (a vertical line), or right to left (a horizontal line). This is defined with the following type:

$$\text{type Line} := \text{TB} \mid \text{RL}$$

ANGLED Blocks contain a line that starts at one side of the block, angles, and exits at another. ANGLED block directions are described as the corner of the square created by the angle within the block. This is defined with the following type:

$$\text{type Corner} := \text{TR} \mid \text{TL} \mid \text{BR} \mid \text{BL}$$

Mazes

A maze is a 2D table of blocks. We will represent each maze as a list of lists of blocks. We will call a list of blocks a “row”, and then a maze is a list of rows. As mentioned in the last problem, our current List type has integer elements, so to express rows and mazes we will define these two new types inductively as follows:

$$\text{type Row} := \text{rnil} \mid \text{rcons}(\text{hd} : \text{Block}, \text{tl} : \text{Row}) \\ \text{type Maze} := \text{mnil} \mid \text{mcons}(\text{hd} : \text{Row}, \text{tl} : \text{Maze})$$

All rows in a maze should have the same length. Mathematically, we define the function $\text{rlen} : \text{Row} \rightarrow \mathbb{N}$ defines the length of a row by:

$$\text{rlen}(\text{rnil}) \quad \quad \quad := 0 \\ \text{rlen}(\text{rcons}(a, L)) \quad := 1 + \text{rlen}(L)$$

Note, however, that our type definitions allow the maze to contain rows of different lengths! It is an additional *invariant* of the Maze type that all rows in each maze must have the same length.

We can also define concatenation of rows. The function $\text{rconcat} : (\text{Row}, \text{Row}) \rightarrow \text{Row}$ is defined by:

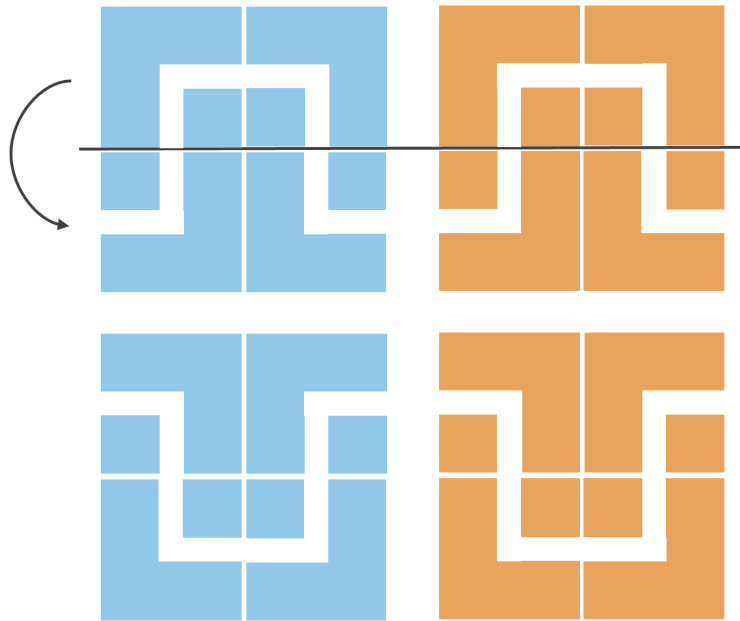
$$\text{rconcat}(\text{rnil}, R) \quad \quad \quad := R \\ \text{rconcat}(\text{rcons}(s, L), R) \quad := \text{rcons}(s, \text{rconcat}(L, R))$$

Task 5 – My Flips are Sealed

[18 pts]

In this problem, we will write a function that “flips a maze vertically, as if spun around a horizontal line through the center”.

Here is an example (the bottom maze is the result of vertically flipping the top maze around the center line):



- (a) The problem definition was in English, so our first step is to formalize it.

Start by writing a mathematical definition of a function “bflip-vert” that flips a **block** vertically.

- (b) Next, we will define a mathematical function “rflip-vert” that flips a **row** vertically.

Let’s start by writing this out in more detail. Let a , b , and c be blocks. Fill in the blanks showing the result of applying rflip-vert to different rows, which we will write as lists of blocks.

Feel free to abbreviate bflip-vert in your answer as “bv”.

rnil _____

rcons(a , rn timer) _____

rcons(a , rcons(b , rn timer)) _____

rcons(a , rcons(b , rcons(c , rn timer)))) _____

...

- (c) Write a mathematical definition of a function rflip-vert using recursion.

- (d) Now, we are ready to define a function “mflip-vert” that flips a **maze** vertically. Note that this operation flips individual rows vertically, but also *reverses* their order!

Again, let’s start by writing this out in more detail. Let u , v , and w be rows. Fill in the blanks showing the result of applying mflip-vert to different mazes, which we will write as lists of rows.

Your answers should use rflip-vert (not bflip-vert), which you can abbreviate as just “ r ”.

mnil _____

mcons(u , mnil) _____

mcons(u , mcons(v , mnil)) _____

mcons(u , mcons(v , mcons(w , mnil))) _____

...

- (e) Write a mathematical definition of a function “mflip-vert”.

Hint: it may be useful to review definition of the function rev, for reversing a list, which is defined in the notes on lists posted on the website. Also, remember that the function mconcat, which concatenates two mazes, is already provided for you.

Homework 4 Code – Solutions

If you have not completed HW4 written. Close this file and finish the written component before starting the coding parts in this worksheet.

To get started, check out the starter code for this assignment:

```
git clone https://gitlab.cs.washington.edu/cse331-24au-materials/hw4-maze.git
```

Navigate to the `hw4-maze` directory and run `npm install --no-audit`. For this assignment, there will be no client side app or server to run. Instead, we have provided (and will ask you to write) unit tests. To run the tests, use the command `npm run test`.

Linter

Starting with this assignment, we will also be using a custom linter, `comfy-tslint`, which checks that your code follows our course-specific coding conventions. The linter is not able to catch every coding convention mistake, so you should write code that looks similar to the examples we go over in class, and refer to the coding conventions [document](#) released last week for a written explanation of our expectations.

We will run the linter on your code through the autograder when you submit on Gradescope, so you should make sure it also passes locally. You can always run the linter in the command line with `npm run lint`, or you can download the recommended [VS Code extension](#) for the linter. The linter warnings will appear as helpful popups while you're coding (similar to the type checker). If you run the command, the output will always have the first two lines listed in the image below, and if your code has any errors, those will be listed below.

```
> cse331-hw-fib@0.0.1 lint
> comfy-tslint --no-mutation src/*.ts,tsx

src/fib.ts:7:32: any type is not allowed
src/index.tsx:6:10: top-level variable declarations must have a type
```

One major check the linter will perform is that you **do not mutate any variables for this assignment**. The following problems will ask you to write tests and translate your math definitions directly to code, neither of which should require any mutation. Our next few assignments will have the same requirement.

Submission

After completing all tasks to follow, submit your solutions on Gradescope. The following completed files should be submitted to **"HW4 Code"**:

```
pell.ts      designs.ts   funcs_test.ts
```

After you submit your work, an autograder will run which verifies you have submitted the correct files, runs the linter, and runs tests (including those you submit, the tests we provide in the starter code, and some additional staff tests). As usual, the autograder is worth points, so you should wait until the autograder completes to make sure it passes, and otherwise resubmit. Meaning you should **leave enough time** to fix possible issues before the deadline. As usual, we will also manually grade your code (including test cases).

Task 6 – Get the Pell out of Dodge

[12 pts]

In this problem, we will translate mathematical definitions for functions into TypeScript code.

Specifically, you will translate math definitions that you wrote in the written Tasks of this assignment. We will treat the math definitions as the imperative specifications for the TypeScript functions, so the translations should be “straight from the spec” –a direct translation.

We have provided tests for these functions based on their correct behavior as described in the English/picture descriptions from the written tasks. You should run these tests to get a good idea of if your implementations are correct using the command `npm run test`.

If the tests fail, indicating a bug in your TypeScript functions, you should fix these bugs to try to get the tests to pass. However, if the bug was due to a mistake in your mathematical definition (rather than a typo or mistake in translation), we will allow you to explain what went wrong to earn some points back on those mistakes and avoid being penalized for not writing a direct translation

To do this, write a brief paragraph (3–4 sentences max is sufficient) within an inline (`//` style) comment at the top of the function explaining how your mathematical definition was incorrect and where the TypeScript function deviates to fix this mistake.

- (a) Translate your mathematical definitions from HW4 Written Task 2, functions `p` and `s`, into TypeScript code in `pell.ts`.

The provided tests for this function are in `pell_test.ts`. These tests and the tests for part (b) are examples of the minimum required tests (according to our course testing guidelines) for these functions.

- (b) Translate your mathematical definitions from HW4 Written Task 4(b), the recursive functions for each maze design, into TypeScript code in `designs.ts`.

You should only translate the *recursive* definitions that you wrote in part **(b)**, you *do not* need to translate the 4×2 versions from part (a). Please maintain the order of the parameters as given in those declarations, even if it deviates from the order in your mathematical definition, as it is required for testing.

The types and helper functions related to mazes (as described in the HW4 Written spec) are translated to TypeScript for you in `maze.ts`; import those to `designs.ts` and use as needed. The provided tests for this function are in `designs_test.ts`.

Task 7 – Test Friends Forever

[12 pts]

Now that you've written some TypeScript code and tested it, it is your turn to write some tests!

In `funcs.ts`, there are 9 functions (fun fact: the first 3 are the same as the functions from HW4 Written Task 1!) that you must write tests for. Your tests should follow the testing requirements we have described in lecture and in our [testing notes summary](#) (also linked on the website "Topics" page).

Write your tests for each function in `funcs_test.ts`.

Additionally, write short labels describing which coverage requirement is met by each test. See the below example function and tests (from this week's section, Task 3):

```
const twice = (L: List): List => {
  if (L.kind === "nil") {
    return nil;
  } else {
    return cons(2 * L.hd, twice(L.tl));
  }
}
```

In test file:

```
it("twice", function() {
  // Statement coverage: [] executes 1st return, [3] executes 2nd
  assert.deepStrictEqual(twice(nil), nil);
  assert.deepStrictEqual(twice(cons(3, nil)), cons(6, nil));

  // Branch coverage: covered above, [] executes 1st branch, [3] executes 2nd

  // Loop/recursion coverage, 0 case: covered above by []
  // Loop/recursion coverage, 1 case: covered above by [3]

  // Loop/recursion coverage: many case
  assert.deepStrictEqual(twice(cons(1, cons(2, cons(3, nil)))),
    cons(2, cons(4, cons(6, nil))));
});
```

Notice how you don't need to add additional tests if previous tests cover multiple requirements, just make sure there are clear comments for the required coverage areas. You are welcome to organize your comments differently than the example or use different wordings, just make sure all necessary details are conveyed.